

정책 기반 공격 탐지 실습

security onion / snort / sgul / zeek

핵심 학습 목표 (Objectives)



1. 현상 관찰 (Observation)

정상적인 사용자 트래픽과
공격 트래픽은
무엇이 다른가?



2. 근거 도출 (Reasoning)

"공격이다"라고 판단할 수 있는
정량적/정성적 근거는
무엇인가?



3. 정책 수립 (Policy Making)

탐지된 공격을 자동 차단할
Snort 룰과 방화벽 정책은
어떻게 작성하는가?

영역별 역할 정의



Attacker (Red)

Kali Linux (VMnet 1)

Nmap, Hydra, Burp Suite 등을 구동하여 공격 시도



Victim (Blue)

Rocky Linux (VMnet 8)

취약한 웹 앱(DVWA) 및 SSH 서비스 운영



Observer (Purple)

Security Onion

Snort(IDS)와 Zeek를 통한 트래픽 감시 및 로그 분석

4대 공격 시나리오 (The 4 Case Files)

#	시나리오 명	도구 (Tool)	핵심 분석 포인트 (Inquiry)
01	정찰 (Reconnaissance)	Nmap	"사람이 1초에 100개의 문(Port)을 두드릴 수 있는가?" → Time Interval, TCP Flags
02	침입 (System Access)	Hydra	"로그인 실패가 기계적으로 반복되는가?" → SSH Failed Log Frequency
03	웹 인증 우회 (Authentication)	Burp Suite	"수천 번의 시도 중 성공한 것은 어떻게 구분하는가?" → HTTP Response Length/Status
04	시스템 명령어 주입 (Command Injection)	Web Browser	"입력값에 들어가면 안 되는 특수문자는 무엇인가?" → Payload Meta Characters (;, `,)

심층 분석: 정찰 및 접근 (Case 1 & 2)

Case 1: Reconnaissance

```
$ nmap -sS -p- 172.16.10.20
```

- 가설: 정상 사용자는 특정 목적지 포트로만 접속한다.
- 이상 징후: 단일 소스 IP에서 1초 미만 간격으로 다수의 목적지 포트(1~65535)로 SYN 패킷 전송.
- 판단 기준: $\text{Time Delta} < 0.01\text{s}$ AND $\text{Distinct Dst Port} > 100$

Case 2: System Access

```
$ hydra -l root -P pass.txt ssh://...
```

- 가설: 사람은 비밀번호 입력에 최소 2~3초가 소요된다.
- 이상 징후: SSH 프로토콜 상에서 'Failed Password' 이벤트가 기계적인 주기로 급증.
- 판단 기준: $\text{Event Count} > 5 \text{ per } 30\text{s}$ (Brute Force Pattern)

심층 분석: 웹 위협 (Case 3 & 4)

Case 3: Authentication

Burp Suite Intruder Attack

- 난제: 수천 개의 '200 OK' 응답 중 실제 로그인 성공을 어떻게 찾는가?
- 해결: 실패 시 반환되는 페이지와 성공 시 페이지의 크기가 다르다.
- 판단 기준: Content-Length Variation (예: 대부분 4500B이나 특정 응답만 5200B)

Case 4: Injection

127.0.0.1; cat /etc/passwd

- 원리: 사용자 입력값을 검증 없이 시스템 셸로 전달할 때 발생.
- 탐지: 정상적인 IP 입력에는 문자가 포함될 수 없다.
- 판단 기준: Payload contains ; (semicolon), | (pipe), ` (backtick)

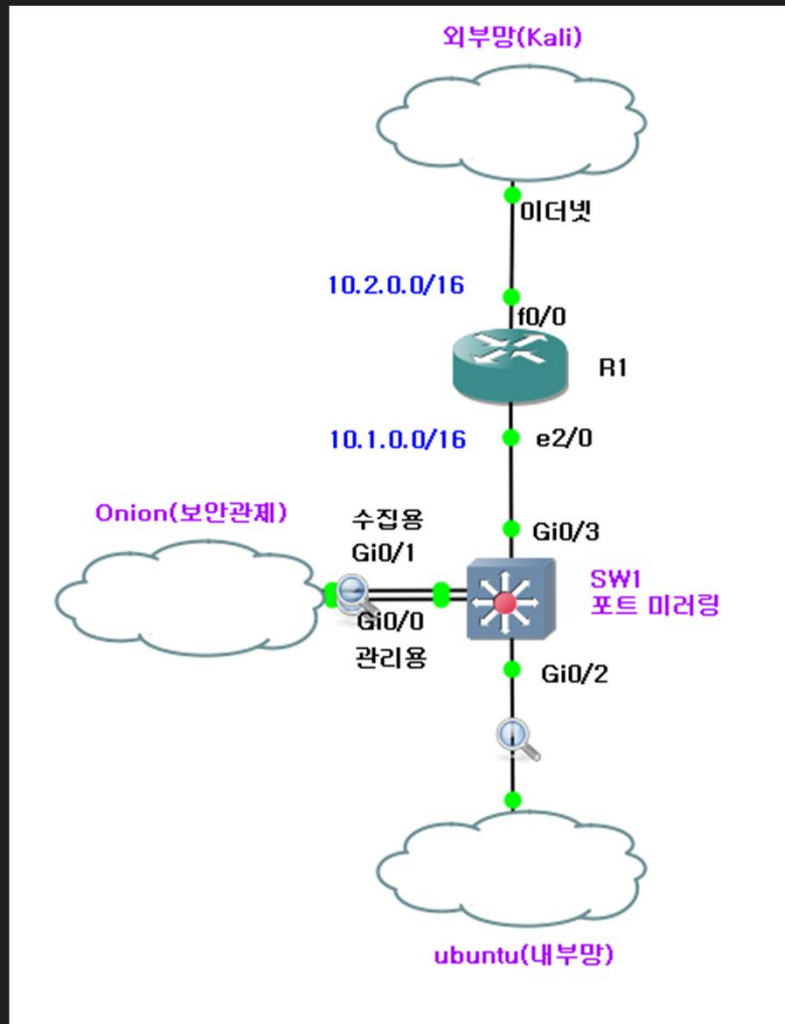
NETWORK SECURITY PRACTICE

실습 환경 구축

Environment Setup

A secure network begins with a correct architecture.

VMware | Kali Linux | Rocky Linux | GNS3



영역별 역할 정의

외부망(Kali)

장비: Kali Linux

네트워크: 10.2.0.2/16 (VMnet 10)

역할: 외부 인터넷에서 기업망을 노리는 해커의 위치. Nmap, Hydra, Burp Suite 등을 구동합니다.

Onion(보안/관제)

장비: Security Onion

네트워크: 10.1.0.10/16 (VMnet 12)

네트워크: (VMnet 13)

역할: 스위치(Switch A)에서 복제(Mirroring)된 모든 트래픽을 수집합니다. Snort(IDS)와 Zeek(로그 분석)가 동작합니다

Ubuntu(내부망)

장비: ubuntu Linux (DVWA 탑재)

네트워크: 10.1.0.20/16 (VMnet 15)

역할: 기업 내부의 웹 서버. 취약점이 존재하는 웹 애플리케이션(DVWA)과 SSH 서비스가 구동 중입니다.

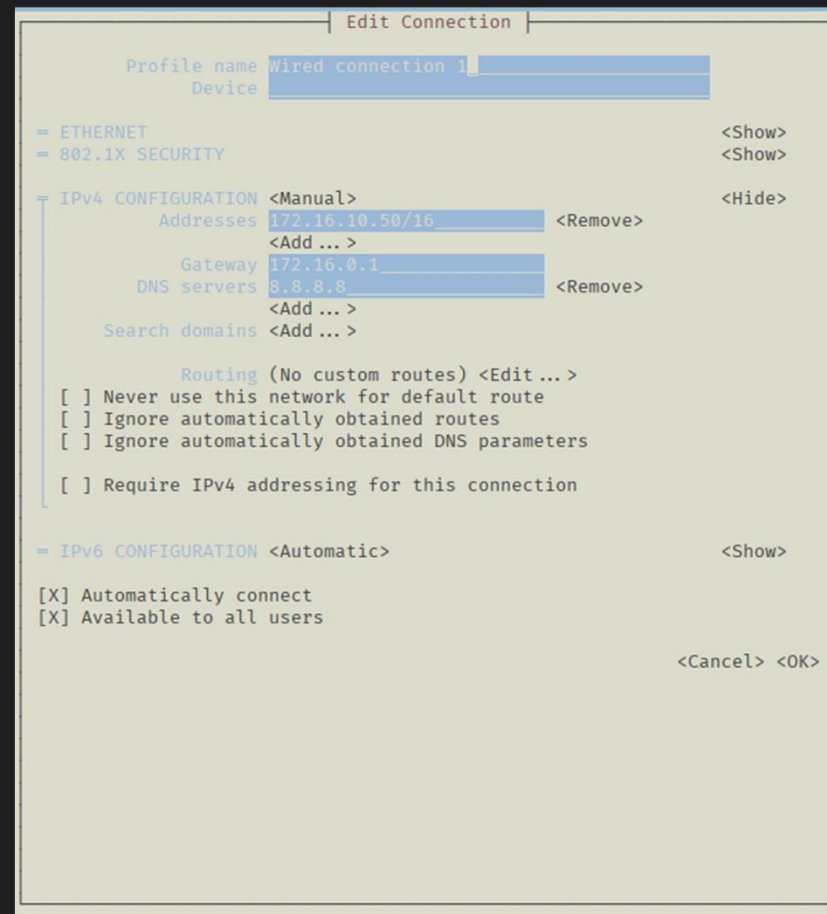
공격자 시스템 준비 (Kali Linux)

- > Network Adapter: VMnet 10

네트워크: 10.2.0.2/16 (VMnet 10)

- > # IP 설정 및 게이트웨이 추가

```
(root@kali)-[~]  
# nmtui
```



필수 공격 도구 점검



Nmap

Port Scanning Tool

```
nmap --version
```



Hydra

Brute Force Tool

```
hydra -h
```



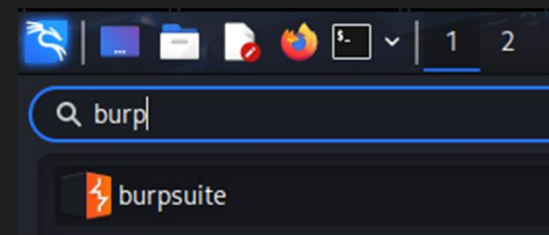
Burp Suite

Web Proxy Tool

```
burpsuite &
```

```
(root@kalikali)-[~]  
# nmap --version  
Nmap version 7.95 ( https://nmap.org )  
Platform: x86_64-pc-linux-gnu  
Compiled with: liblua-5.4.7 openssl-3.5.2 libs  
Compiled without:  
Available nsock engines: epoll poll select
```

```
(root@kalikali)-[~]  
# hydra -h  
Hydra v9.5 (c) 2023 by van Hauser/TH  
legal purposes (this is non-binding,
```



피해자 시스템 (Rocky)

- > **Network Adapter:** VMnet 15 (NAT)
- > **Role:** Trusted Zone (내부망 웹서버)
- > **IP Address:** 10.1.0.20/16

서비스 활성화 확인

```
root@dk:~# systemctl status bind9
• named.service - BIND Domain Name Server
  Loaded: loaded (/usr/lib/systemd/system/named.service; vendor preset: enabled)
  Active: active (running) since Mon 2025-12-01 15:00:00 KST; 1min 1s ago
  Truncated output...
```

```
network:
  version: 2
  ethernets:
    ens33:
      addresses:
        - "10.1.0.20/16"
      nameservers:
        addresses:
          - 8.8.8.8
        search: []
      routes:
        - to: "default"
          via: "10.1.0.1"
```

```
root@dk:~# netplan apply
```

보안/관제(Onion)

- > Network Adapter: VMnet 12
- > IP Address: 10.1.0.10/16

```
sudo apt-get install open-vm-tools-desktop fuse
```

관리자 모드

```
cat > /etc/network/interfaces <<'EOF'
auto lo
iface lo inet loopback
```

```
auto ens33
iface ens33 inet static
address 10.1.0.10
gateway 10.1.0.1
netmask 255.255.0.0
dns-nameservers 8.8.8.8
dns-domain sinu.com
```

```
auto ens34
iface ens34 inet manual
up ip link set $IFACE promisc on arp off up
down ip link set $IFACE promisc off down
post-up for i in rx tx sg tso ufo gso gro lro; do ethtool -K $IFACE $i off; done
post-up echo 1 > /proc/sys/net/ipv6/conf/$IFACE/disable_ipv6
EOF
```

```
sudo ip addr del 172.16.10.88/16 dev ens33
sudo ifdown ens33 && sudo ifup ens33
```

GNS3 인프라 및 라우터 설정

Cloud Node Mapping

GNS3 Cloud 노드를 VMware 어댑터와 매핑합니다.

- > **Cloud-Ext:** VMnet 10 (Attacker)
- > **Cloud-Int:** VMnet 15 (Internal)
- > **Onion:** Vmnet 12 (관리용)
- > **onion:** VMnet 13 (탐지용)

Router Gateway Config

```
R1(config)#int f0/0
```

```
R1(config-if)#ip add 172.16.255.1 255.255.0.0
```

```
R1(config-if)#no sh
```

```
R1(config-if)#exit
```

```
R1(config)#ip route 0.0.0.0 0.0.0.0 172.16.0.1
```

```
R1(config)#int e2/0
```

```
R1(config-if)#ip add ip add 10.1.0.1 255.255.0.0
```

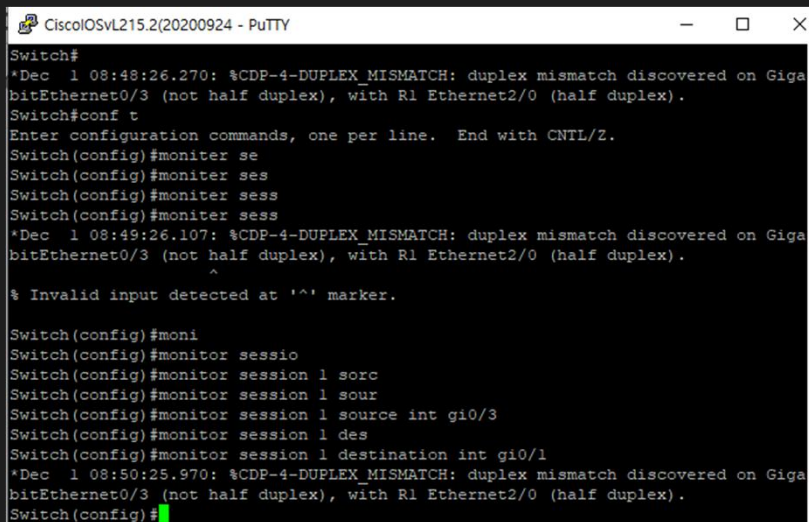
```
R1(config-if)#no sh
```

핵심: 스위치 및 포트 미러링 (SPAN)

Security Onion(IDS)이 패킷을 감청할 수 있도록 **SPAN(Switch Port Analyzer)**을 설정합니다.

Switch(config)#monitor session 1 source int gi0/3 both

Switch(config)#monitor session 1 destination int gi0/1



```
CiscoIOSvL215.2(20200924) - PuTTY
Switch#
*Dec 1 08:48:26.270: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on Giga
bitEthernet0/3 (not half duplex), with R1 Ethernet2/0 (half duplex).
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#monitor se
Switch(config)#monitor ses
Switch(config)#monitor sess
Switch(config)#monitor sess
*Dec 1 08:49:26.107: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on Giga
bitEthernet0/3 (not half duplex), with R1 Ethernet2/0 (half duplex).
^
% Invalid input detected at '^' marker.

Switch(config)#moni
Switch(config)#monitor sessio
Switch(config)#monitor session 1 sourc
Switch(config)#monitor session 1 sour
Switch(config)#monitor session 1 source int gi0/3
Switch(config)#monitor session 1 des
Switch(config)#monitor session 1 destination int gi0/1
*Dec 1 08:50:25.970: %CDP-4-DUPLEX_MISMATCH: duplex mismatch discovered on Giga
bitEthernet0/3 (not half duplex), with R1 Ethernet2/0 (half duplex).
Switch(config)#
```

[시나리오 1]

NMAP PORT SCANNING

Action, Observation, Deduction, Definition

PHASE 1: ACTION (공격 수행)

KALI LINUX 공격 준비

터미널에서 SYN Stealth Scan을 수행합니다.

1. 대상 생존 여부 확인

```
ping -c 3 10.1.0.20
```

2. 공격 수행 (SYN Stealth Scan)

-sS: SYN 스캔 (완전한 연결을 맺지 않음)

-T4: 빠른 속도 (0~5 단계 중 4 단계)

-p 1-100: 1 번부터 100 번 포트까지만 스캔 (빠른 실습을 위해)

```
nmap -sS -T4 -p 1-100 10.1.0.20
```

```
(root@kalikali)-[~]
# ping -c 3 10.1.0.20
PING 10.1.0.20 (10.1.0.20) 56(84) bytes of data.
64 bytes from 10.1.0.20: icmp_seq=1 ttl=63 time=34.8 ms
From 172.16.0.1 icmp_seq=2 Redirect Host(New nexthop: 172.16.255.1)
64 bytes from 10.1.0.20: icmp_seq=2 ttl=63 time=17.8 ms

— 10.1.0.20 ping statistics —
2 packets transmitted, 2 received, +1 errors, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 17.816/26.320/34.824/8.504 ms
```

```
(root@kalikali)-[~]
# nmap -sS -T4 -p 1-100 10.1.0.20
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-01 01:01 PST
Nmap scan report for 10.1.0.20
Host is up (0.17s latency).
Not shown: 96 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds
```


PHASE 1: ACTION (결과 확인)

NMAP 스캔 결과

공격자는 타겟 시스템의 열려있는 포트(서비스)를 식별합니다.

PORT	STATE	SERVICE
21/tcp	open	ftp
22/tcp	open	ssh
53/tcp	open	domain
80/tcp	open	http

🎯 공격자의 획득 정보

- 타겟 IP: 10.1.0.10
- 활성화된 서비스: SSH (원격 접속), HTTP (웹 서버)
- 다음 단계: 해당 서비스의 취약점 탐색 가능

PHASE 2: OBSERVATION (WIRESHARK 분석)

✔ 정상 접속 (3-WAY HANDSHAKE)

1. SYN (접속 요청)
2. SYN, ACK (승인)
3. ACK (연결 수립)

정상적인 연결은 로그가 남으며, 세션이 맺어집니다.

🚫 NMAP SYN SCAN (-SS)

1. SYN (문 열렸니?)
2. SYN, ACK (응, 열렸어)
3. RST (아냐, 끊어!)

로그를 남기지 않기 위해 연결을 강제로 종료합니다 (Half-open).

159	244.515208	172.16.10.1	10.1.0.20	TCP	66	50403 → 80	[SYN] Seq=0
160	244.516375	172.16.10.1	10.1.0.20	TCP	66	50405 → 80	[SYN] Seq=0
161	244.525566	10.1.0.20	172.16.10.1	TCP	66	80 → 50403	[SYN, ACK] Seq=1
162	244.527853	10.1.0.20	172.16.10.1	TCP	66	80 → 50405	[SYN, ACK] Seq=1
163	244.575177	172.16.10.1	10.1.0.20	TCP	60	50403 → 80	[ACK] Seq=1
164	244.576435	172.16.10.1	10.1.0.20	TCP	60	50405 → 80	[ACK] Seq=1

232	304.140802	172.16.10.50	10.1.0.20	TCP	60	58526 → 53	[SYN] Seq=0
233	304.141160	10.1.0.20	172.16.10.50	TCP	60	53 → 58526	[SYN, ACK] Seq=0
234	304.147617	172.16.10.50	10.1.0.20	TCP	60	58526 → 76	[SYN] Seq=0
235	304.147853	10.1.0.20	172.16.10.50	TCP	60	76 → 58526	[RST, ACK] Seq=0
236	304.148876	172.16.10.50	10.1.0.20	TCP	60	58526 → 78	[SYN] Seq=0
237	304.149077	10.1.0.20	172.16.10.50	TCP	60	78 → 58526	[RST, ACK] Seq=0
238	304.154610	172.16.10.50	10.1.0.20	TCP	60	58526 → 96	[SYN] Seq=0
239	304.154838	10.1.0.20	172.16.10.50	TCP	60	96 → 58526	[RST, ACK] Seq=0
240	304.157789	172.16.10.50	10.1.0.20	TCP	60	58526 → 19	[SYN] Seq=0
241	304.158059	10.1.0.20	172.16.10.50	TCP	60	19 → 58526	[RST, ACK] Seq=0
242	304.164401	172.16.10.50	10.1.0.20	TCP	60	58526 → 22	[RST] Seq=1
243	304.165485	172.16.10.50	10.1.0.20	TCP	60	58526 → 21	[RST] Seq=1

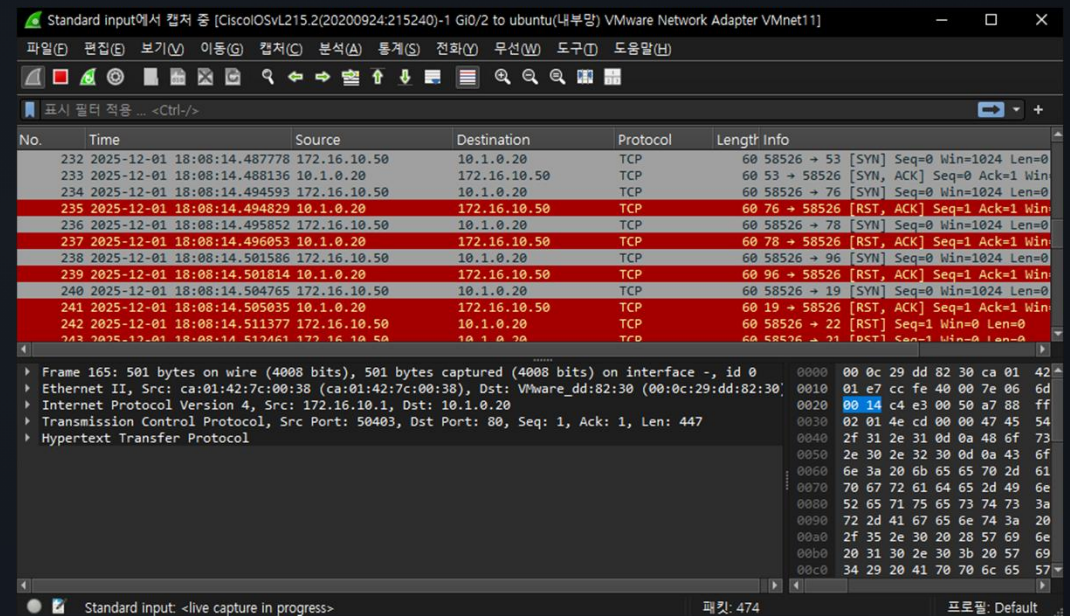
PHASE 2: 관찰 포인트 (HALF-OPEN & INTERVAL)

HALF-OPEN SCAN

공격자는 RST(Reset) 패킷을 보내 연결을 즉시 끊어버립니다. 이를 통해 3-Way Handshake를 완성하지 않고, 로그 기록을 회피하려 합니다.

시간 간격 (TIME INTERVAL)

Wireshark의 Time 컬럼을 확인하면, 사람이 수동으로 할 수 없는 속도(0.001초 단위)로 패킷이 쏟아지는 것을 볼 수 있습니다.



PHASE 3: DEDUCTION (추론 및 판단)

"무엇을 기준으로 탐지할 것인가?"

[RST]

RST 패킷은 네트워크 불안정 시에도 발생하므로 단독 증거로는 부족합니다. SYN 패킷의 빈도가 더 중요합니다.

2025-12-01 18:08:14.	2025-12-01 18:08:14.	2025-12-01 18:08:14.
2025-12-01 18:08:14.	2025-12-01 18:08:14.	2025-12-01 18:08:14.
2025-12-01 18:08:14.	2025-12-01 18:08:14.	2025-12-01 18:08:14.
2025-12-01 18:08:14.	2025-12-01 18:08:14.	2025-12-01 18:08:14.
2025-12-01 18:08:14.	2025-12-01 18:08:14.	2025-12-01 18:08:14.

정상적인 웹 서핑과 달리, "서로 다른 포트"를 향해 "짧은 시간" 동안 찌르는 행위는 명확한 공격 징후입니다.

▲ 공격 판단 기준 (Criteria)

"동일 출발지 IP + 짧은 시간(Time Window) + 다수의 서로 다른 포트(Distinct Ports) + SYN 패킷"

PHASE 4: DEFINITION (SNORT 탐지 룰)

```
sudo vi /etc/nsm/rules/local.rules
sudo cp /etc/nsm/rules/local.rules /etc/nsm/rules/downloaded.rules
sudo nsm_sensor_ps-restart --only-snort-alert
```

도출된 기준을 바탕으로 IDS(Snort) 탐지 룰을 작성합니다.

Nmap Port Scan (Nmap)	공격 (Attack)	alert tcp any any -> 10.1.0.20 any (msg:"Nmap - SYN Scan Detected"; flags:S; flow:stateless; detection_filter:track by_src, count 20, seconds 2; sid:40002;)	짧은 시간(2초) 내에 특정 호스트가 20개 이상의 포트를 찌르는 행위는 전형적인 Nmap 스캔(-sS 등)이므로 탐지
alert		조건에 맞으면 경고를 발생시키고 로그를 남깁니다.	탐지 모드
any (목적지 포트)		목적지 포트가 무엇이든 상관없음 (모든 포트 감시).	포트 스캔 탐지용
flags:S		오직 SYN 패킷만 감지합니다.	Nmap -sS 옵션 대응
flow:stateless		TCP 상태(연결 수립 여부 등)를 추적하지 않고, 단일 패킷 자체만 봅니다.	스캔은 연결을 맺지 않으므로 성능상 유리함
detection_filter		이 룰의 핵심 (임계치 설정)	빈도 기반 탐지
track by_src		"누가" 공격하는지(출발지 IP 기준) 카운트를 셉니다.	
count 20		20번째 패킷부터 탐지합니다.	
seconds 2		2초라는 시간 윈도우(창) 내에서 셉니다.	

PHASE 4: DEFINITION (SNORT 탐지 룰)

RT	981	sinu-virtu...	3.187	2025-12-04 03:27:50	172.16.10.50	57196	10.1.0.20	80	6	Snort Alert [1:1040002:0]
----	-----	---------------	-------	---------------------	--------------	-------	-----------	----	---	---------------------------

```
alert tcp any any -> 10.1.0.20 any (msg:"Nmap - SYN Scan Detected"; flags:S; flow:stateless;  
detection_filter:track by_src, count 20, seconds 2; sid:1040002;)  
/nsm/server_data/securityonion/rules/sinu-virtual-machine-ens34-1/local.rules: Line 1
```

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.10.50	10.1.0.20	TCP	60	57196 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.005051	10.1.0.20	172.16.10.50	TCP	60	80 → 57196 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=14...
3	0.656804	172.16.10.50	10.1.0.20	TCP	60	57196 → 80 [RST] Seq=1 Win=0 Len=0

cd /nsm/bro/logs/current ← Zeek 로그 남는 위치
여기서 conn.log(네트워크 모든 연결 로그) 확인

PHASE 4: DEFINITION (ROUTER ACL 차단) <- 아직 적용 L L

탐지된 공격자 IP(172.16.10.50)를 라우터에서 물리적으로 차단합니다.

```
access-list 100 deny ip 192.168.100.0 0.0.0.255 any
```

```
access-list 100 permit ip any any
```

```
Interface f0/0
```

```
Ip access-group 100 in
```

결과: 다시 Nmap 스캔 시 모든 포트가 Filtered 되거나 Timeout 발생.

SUMMARY (요약 및 결론)



공격 기법

흔적을 최소화하기 위해
Half-open (SYN Scan)
방식을 사용

탐지 원리

비정상적인 트래픽 발생 빈도
(Threshold)는 숨길 수 없음



대응 방안

IDS(Snort)로 감지하고,
FW(ACL)로 차단

Next Step: [시나리오 2] 문을 부수고 들어오는 Brute Force 공격

[시나리오 2]

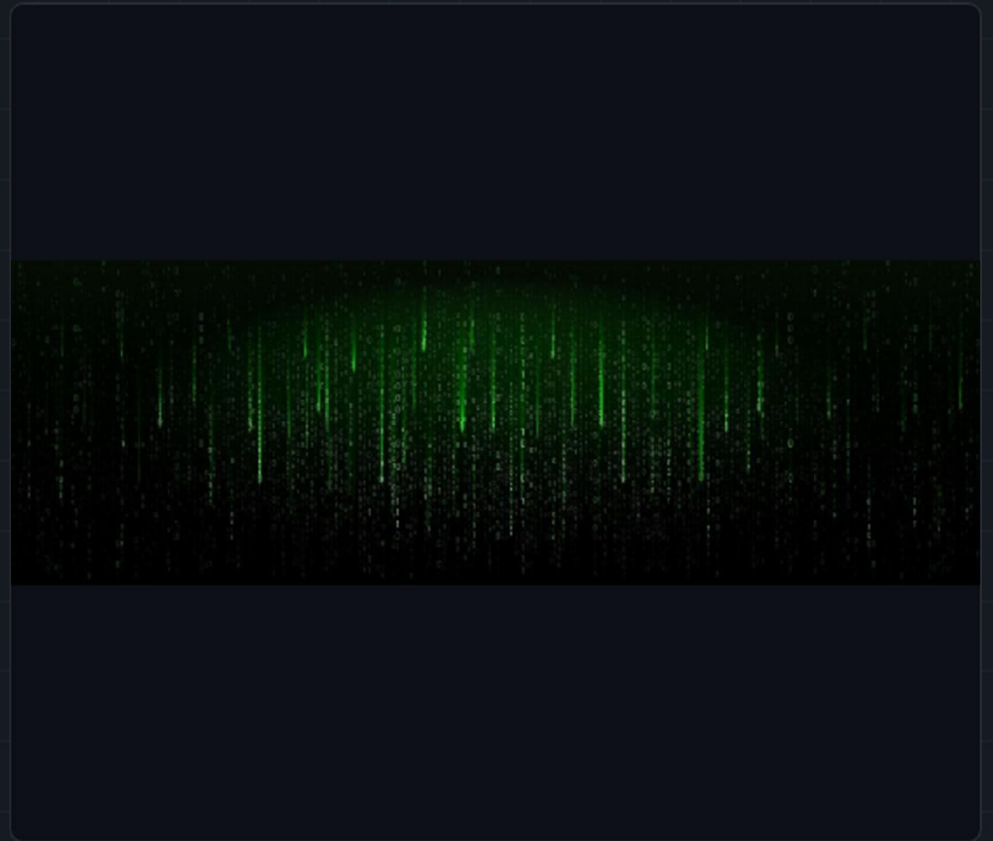
SSH Brute Force

무차별 대입 공격 (Brute Force)

가장 원초적이지만 강력한 공격

시스템 관리자 권한(root)을 탈취하기 위한 전통적인 공격 방식입니다.

- 목표: 정확한 비밀번호를 맞출 때까지 가능한 모든 조합을 시도
- 특징: 다수의 로그인 실패 로그 발생, 높은 네트워크 트래픽 유발
- 도구: Hydra, Medusa, Ncrack 등



Phase 1: Action (사전 준비)

1. 사전 파일(Dictionary) 준비

공격자(Kali)는 기계적인 접속 시도를 위해 비밀번호 목록을 생성합니다.

```
(root@kali)-[~]  
# ls  
Desktop  
Documents  
Downloads  
hydra.restore  
Music  
password.txt  
Pictures  
Public  
rockyou.txt  
rockyou.txt.tar.gz  
Templates  
Videos
```

Cd /usr/share/wordlists <-kali rockyou 기본 파일 위치
Sudo gzip -d rockyou.txt.gz <- 압축 해제 후
Mv 명령어를 통해 원하는 위치로 rockyou.txt 파일만 이동

전략

- 사람들이 흔히 사용하는 비밀번호를 목록화
- 관리자 계정(root)을 주요 타겟으로 설정
- 수천, 수만 개의 조합을 파일로 저장하여 자동화 공격 준비

2. 공격 수행 (Hydra)

```
hydra -l root -P rockyou.txt 10.1.0.20 ssh
```

Hydra는 매우 빠른 속도로 병렬 로그인 시도를 수행하는 도구입니다.

옵션	설명	예시 값
-l	로그인할 단일 ID 지정	root
-P	비밀번호 사전 파일 경로	pass.txt
-t	동시 실행 스레드(Task) 수	4 (너무 높으면 연결 끊김)
-vV	상세 진행 과정 출력 (Verbose)	상세 로그 확인용
ssh://	대상 프로토콜 및 IP	ssh://172.16.10.20

Phase 2: Observation

(로그 분석)

관찰 포인트 1: 실패 로그

피해자 시스템(Rocky Linux)에서 인증 로그를 실시간으로 확인합니다.

```
cat /var/log/secure <-rocky
cat /var/log/auth.log <-ubuntu
```

- 정상 사용자: 3~5초에 한 번 실수
- 공격자: 1초에 4~5줄 이상의 Failed password 로그가 폭포수처럼 발생

```
2025-12-03T03:23:06.141217+00:00 dk sshd-session[2993]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.142300+00:00 dk sshd-session[2988]: error: maximum authentication attempts exceeded for root from 172.16.10.50 port 46614
2025-12-03T03:23:06.145964+00:00 dk sshd-session[2988]: Disconnecting authenticating user root 172.16.10.50 port 46614: Too many authentication
h)
2025-12-03T03:23:06.147117+00:00 dk sshd-session[2988]: PAM 5 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=172.16.
2025-12-03T03:23:06.147426+00:00 dk sshd-session[2988]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.152487+00:00 dk sshd[2975]: drop connection #5 from [172.16.10.50]:36812 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.159005+00:00 dk sshd-session[2989]: error: maximum authentication attempts exceeded for root from 172.16.10.50 port 46630
2025-12-03T03:23:06.159210+00:00 dk sshd-session[2989]: Disconnecting authenticating user root 172.16.10.50 port 46630: Too many authenticatio
h)
2025-12-03T03:23:06.159561+00:00 dk sshd-session[2989]: PAM 5 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=172.16.
2025-12-03T03:23:06.159705+00:00 dk sshd-session[2989]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.169838+00:00 dk sshd-session[2992]: error: maximum authentication attempts exceeded for root from 172.16.10.50 port 46648
2025-12-03T03:23:06.170486+00:00 dk sshd-session[2992]: Disconnecting authenticating user root 172.16.10.50 port 46648: Too many authenticatio
h)
2025-12-03T03:23:06.171065+00:00 dk sshd-session[2992]: PAM 5 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=172.16.
2025-12-03T03:23:06.171198+00:00 dk sshd-session[2992]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.182158+00:00 dk sshd-session[2998]: error: maximum authentication attempts exceeded for root from 172.16.10.50 port 46714
2025-12-03T03:23:06.190513+00:00 dk sshd-session[2997]: error: maximum authentication attempts exceeded for root from 172.16.10.50 port 46688
2025-12-03T03:23:06.191334+00:00 dk sshd-session[2997]: Disconnecting authenticating user root 172.16.10.50 port 46688: Too many authenticatio
h)
2025-12-03T03:23:06.191404+00:00 dk sshd-session[2997]: PAM 5 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=172.16.
2025-12-03T03:23:06.191555+00:00 dk sshd-session[2997]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.191620+00:00 dk sshd-session[2998]: Disconnecting authenticating user root 172.16.10.50 port 46714: Too many authenticatio
h)
2025-12-03T03:23:06.191694+00:00 dk sshd-session[2998]: PAM 5 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=172.16.
2025-12-03T03:23:06.191751+00:00 dk sshd-session[2998]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.214618+00:00 dk sshd-session[2996]: error: maximum authentication attempts exceeded for root from 172.16.10.50 port 46678
2025-12-03T03:23:06.217354+00:00 dk sshd-session[2996]: Disconnecting authenticating user root 172.16.10.50 port 46678: Too many authenticatio
h)
2025-12-03T03:23:06.228087+00:00 dk sshd[2975]: drop connection #1 from [172.16.10.50]:36816 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.255121+00:00 dk sshd-session[2996]: PAM 5 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=172.16.
2025-12-03T03:23:06.255159+00:00 dk sshd-session[2996]: PAM service(sshd) ignoring max retries; 6 > 3
2025-12-03T03:23:06.286249+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36820 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.290524+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36828 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.315358+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36838 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.316309+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36852 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.343257+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36864 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.383077+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36876 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.396094+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36884 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.435431+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36898 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.449685+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36906 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.456904+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36912 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.481250+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36914 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.489206+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36930 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.499171+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36936 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.521930+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36952 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.543573+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36956 on [10.1.0.20]:22 penalty: failed authentication
2025-12-03T03:23:06.548253+00:00 dk sshd[2975]: drop connection #0 from [172.16.10.50]:36960 on [10.1.0.20]:22 penalty: failed authentication
```

네트워크 패킷 분석 (Wireshark)

관찰 포인트 2: SSH 프로토콜

Security Onion 또는 Wireshark를 통해 트래픽을 분석합니다.



TCP Flags: 수많은 SYN 패킷과 함께 데이터 전송을 의미하는 PSH, ACK 패킷이 뒤섞여 관찰됩니다.

Protocol Hierarchy: 전체 트래픽 중 SSH 프로토콜이 비정상적으로 높은 비율을 차지합니다.

083143	172.16.10.50	10.1.0.20	SSHv2	150 Client: Encrypted packet (len=84)
105962	10.1.0.20	172.16.10.50	TCP	66 22 → 46608 [ACK] Seq=2058 Ack=1576 Win=6
107850	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
113075	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
113683	172.16.10.50	10.1.0.20	SSHv2	150 Client: Encrypted packet (len=84)
114348	172.16.10.50	10.1.0.20	SSHv2	150 Client: Encrypted packet (len=84)
120985	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
126402	10.1.0.20	172.16.10.50	TCP	66 22 → 46604 [ACK] Seq=2058 Ack=1568 Win=6
126536	10.1.0.20	172.16.10.50	TCP	66 22 → 46598 [ACK] Seq=2058 Ack=1568 Win=6
128192	172.16.10.50	10.1.0.20	SSHv2	150 Client: Encrypted packet (len=84)
135783	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
139561	10.1.0.20	172.16.10.50	TCP	66 22 → 46636 [ACK] Seq=2058 Ack=1568 Win=6
144322	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
146124	172.16.10.50	10.1.0.20	SSHv2	142 Client: Encrypted packet (len=76)
146325	10.1.0.20	172.16.10.50	TCP	66 22 → 46660 [ACK] Seq=2058 Ack=1560 Win=6
147276	172.16.10.50	10.1.0.20	SSHv2	142 Client: Encrypted packet (len=76)
147800	10.1.0.20	172.16.10.50	TCP	66 22 → 46614 [ACK] Seq=2058 Ack=1568 Win=6
163841	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
165373	10.1.0.20	172.16.10.50	SSHv2	118 Server: Encrypted packet (len=52)
166960	172.16.10.50	10.1.0.20	SSHv2	142 Client: Encrypted packet (len=76)
167405	10.1.0.20	172.16.10.50	TCP	66 22 → 46630 [ACK] Seq=2058 Ack=1560 Win=6
176070	172.16.10.50	10.1.0.20	SSHv2	142 Client: Encrypted packet (len=76)

10.1.0.20	172.16.10.50	TCP	74 22 → 36884 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0
10.1.0.20	172.16.10.50	TCP	92 22 → 36816 [PSH, ACK] Seq=1 Ack=24 Win=65152 Len=0
10.1.0.20	172.16.10.50	TCP	66 22 → 36816 [RST, ACK] Seq=27 Ack=24 Win=65152 Len=0
10.1.0.20	172.16.10.50	TCP	66 22 → 46678 [FIN, ACK] Seq=2134 Ack=1576 Win=642
172.16.10.50	10.1.0.20	TCP	66 36812 → 22 [ACK] Seq=24 Ack=27 Win=64256 Len=0
172.16.10.50	10.1.0.20	TCP	66 36812 → 22 [FIN, ACK] Seq=24 Ack=28 Win=64256 Len=0
10.1.0.20	172.16.10.50	TCP	60 22 → 36812 [RST] Seq=27 Win=0 Len=0
10.1.0.20	172.16.10.50	TCP	66 [TCP Retransmission] 22 → 46630 [FIN, ACK] Seq=
10.1.0.20	172.16.10.50	TCP	60 22 → 36812 [RST] Seq=28 Win=0 Len=0
172.16.10.50	10.1.0.20	TCP	66 [TCP Retransmission] 36796 → 22 [FIN, ACK] Seq=
172.16.10.50	10.1.0.20	TCP	66 36820 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TS
10.1.0.20	172.16.10.50	TCP	66 [TCP Retransmission] 22 → 46648 [FIN, ACK] Seq=
172.16.10.50	10.1.0.20	TCP	66 36828 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TS
10.1.0.20	172.16.10.50	TCP	60 22 → 36796 [RST] Seq=28 Win=0 Len=0
10.1.0.20	172.16.10.50	SSHv2	92
10.1.0.20	172.16.10.50	TCP	66 22 → 36820 [FIN, ACK] Seq=27 Ack=1 Win=65280 Le
10.1.0.20	172.16.10.50	SSHv2	92
10.1.0.20	172.16.10.50	TCP	66 22 → 36828 [FIN, ACK] Seq=27 Ack=1 Win=65280 Le
172.16.10.50	10.1.0.20	TCP	66 46648 → 22 [FIN, ACK] Seq=1560 Ack=2134 Win=689
10.1.0.20	172.16.10.50	TCP	66 [TCP Retransmission] 22 → 46714 [FIN, ACK] Seq=
10.1.0.20	172.16.10.50	TCP	66 22 → 46648 [ACK] Seq=2135 Ack=1561 Win=64256 Le
10.1.0.20	172.16.10.50	TCP	66 [TCP Retransmission] 22 → 46688 [FIN, ACK] Seq=

Phase 3: Deduction (추론 및 판단)

"로그인 실패는 죄가 아니지만, 기계적인 반복은 공격입니다."
정상 사용자와 공격자를 구분하는 임계값(Threshold)을 설정해야 합니다.

20s

Time Window

20초 이내에 발생하는

×

5+

Attempts

5회 이상의 연결 시도

결론: 동일 IP에서 20초 내 5회 이상 SYN 발생 시 공격으로 간주

PHASE 4: DEFINITION (SNORT 탐지 룰)

```
sudo vi /etc/nsm/rules/local.rules
sudo cp /etc/nsm/rules/local.rules /etc/nsm/rules/downloaded.rules
sudo nsm_sensor_ps-restart --only-snort-alert
```

도출된 기준을 바탕으로 IDS(Snort) 탐지 룰을 작성합니다.

SSH Brute Force (Hydra)	공격 (Attack)	alert tcp \$EXTERNAL_NET any -> \$HOME_NET 22 (msg:"Hydra - SSH Brute Force Detected"; flags:S; flow:stateless; detection_filter:track by_src, count 5, seconds 20; sid:20002; rev:1;)	Hydra 툴 특성상 짧은 시간 내에 다수의 로그인 시도(SYN 연결)가 발생함. 20초 내 5회 이상 접속 시도시 무차별 대입 공격으로 판단
flags:S	오직 SYN 패킷만 감지합니다.		
track by_src	"누가" 공격하는지(출발지 IP 기준) 카운트를 셉니다.		
count 5	5번째 패킷부터 탐지합니다.		
seconds 20	20초라는 시간 내에서 셉니다.		

PHASE 4: DEFINITION (SNORT 탐지 룰)

172.16.10.50_34236_10.1.0.20_22-6.raw

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/> Expression...

No.	Time	Source	Destination	Protocol	Length	Info
28	5.557074	10.1.0.20	172.16.10.50	SSHv2	118	Server: Encrypted packet (len=52)
29	5.584068	172.16.10.50	10.1.0.20	SSHv2	150	Client: Encrypted packet (len=84)
30	5.589123	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [ACK] Seq=1850 Ack=1264 Win=64256 L...
31	9.149795	10.1.0.20	172.16.10.50	SSHv2	118	Server: Encrypted packet (len=52)
32	9.169958	172.16.10.50	10.1.0.20	SSHv2	150	Client: Encrypted packet (len=84)
33	9.174440	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [ACK] Seq=1902 Ack=1348 Win=64256 L...
34	10.954195	10.1.0.20	172.16.10.50	SSHv2	118	Server: Encrypted packet (len=52)
35	10.988064	172.16.10.50	10.1.0.20	SSHv2	142	Client: Encrypted packet (len=76)
36	10.992432	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [ACK] Seq=1954 Ack=1424 Win=64256 L...
37	14.558060	10.1.0.20	172.16.10.50	SSHv2	118	Server: Encrypted packet (len=52)
38	14.587339	172.16.10.50	10.1.0.20	SSHv2	150	Client: Encrypted packet (len=84)
39	14.592158	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [ACK] Seq=2006 Ack=1508 Win=64256 L...
40	16.372481	10.1.0.20	172.16.10.50	SSHv2	118	Server: Encrypted packet (len=52)
41	16.404697	172.16.10.50	10.1.0.20	SSHv2	142	Client: Encrypted packet (len=76)
42	16.409275	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [ACK] Seq=2058 Ack=1584 Win=64256 L...
43	18.202100	10.1.0.20	172.16.10.50	SSHv2	142	Server: Encrypted packet (len=76)
44	18.203578	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [FIN, ACK] Seq=2134 Ack=1584 Win=64...
45	18.237150	172.16.10.50	10.1.0.20	TCP	66	34236 → 22 [FIN, ACK] Seq=1584 Ack=2134 Win=68...
46	18.239166	172.16.10.50	10.1.0.20	TCP	66	34236 → 22 [ACK] Seq=1585 Ack=2135 Win=68992 L...
47	18.259286	10.1.0.20	172.16.10.50	TCP	66	22 → 34236 [ACK] Seq=2135 Ack=1585 Win=64256 L...

▶ Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
▶ Ethernet II, Src: ca:01:42:7c:00:38, Dst: 00:0c:29:dd:82:30
▶ Internet Protocol Version 4, Src: 172.16.10.50, Dst: 10.1.0.20
▶ Transmission Control Protocol, Src Port: 34236, Dst Port: 22, Seq: 0, Len: 0

0000 00 0c 29 dd 82 30 ca 01 42 7c 00 38 08 00 45 00 ..).0.. B|·8·E·
0010 00 3c fe a3 40 00 3f 06 7c c1 ac 10 0a 32 0a 01 ·<·@·?· |···2·

172.16.10.50_34236_10.1.0.20_22-6.raw Packets: 47 · Displayed: 47 (100.0%) Profile: Default

시스템 레벨의 방어 (Hardening)

네트워크 장비뿐만 아니라 서버 자체 설정을 강화하는 것도 중요합니다.
/etc/ssh/sshd_config 파일을 수정하여 보안을 강화합니다.



MaxAuthTries 3

최대 인증 시도 횟수를 3회로 제한하여 무제한 대입 공격을 차단합니다.



PermitRootLogin no

Root 계정 직접 로그인을 차단합니다. 공격자가 ID를 알아내는 과정을 한 단계 더 어렵게 만듭니다.

SCENARIO 3

웹 인증 우회:

HTTP Brute Force

Using Burp Suite for Web Form Attacks

| Phase 1: Action - 공격 준비



Burp Suite 설정

Kali Linux에서 Firefox 프록시를 10.1.0.20:8080으로 설정하고 Burp Suite를 실행하여 Intercept is on 상태를 확인합니다.



패킷 캡처 (Capture)

DVWA 로그인 페이지에서 임의의 ID/PW(admin/1234)를 입력하고 로그인 버튼을 눌러 요청 패킷을 가로챍니다.



Intruder 전송

캡처된 패킷을 확인한 후 우클릭하여 'Send to Intruder (Ctrl + I)'를 선택해 공격 도구로 전송합니다.

Intruder 페이로드 구성

> Attack Type: Sniper

단일 파라미터에 대해 값을 대입하는 방식을 선택합니다.

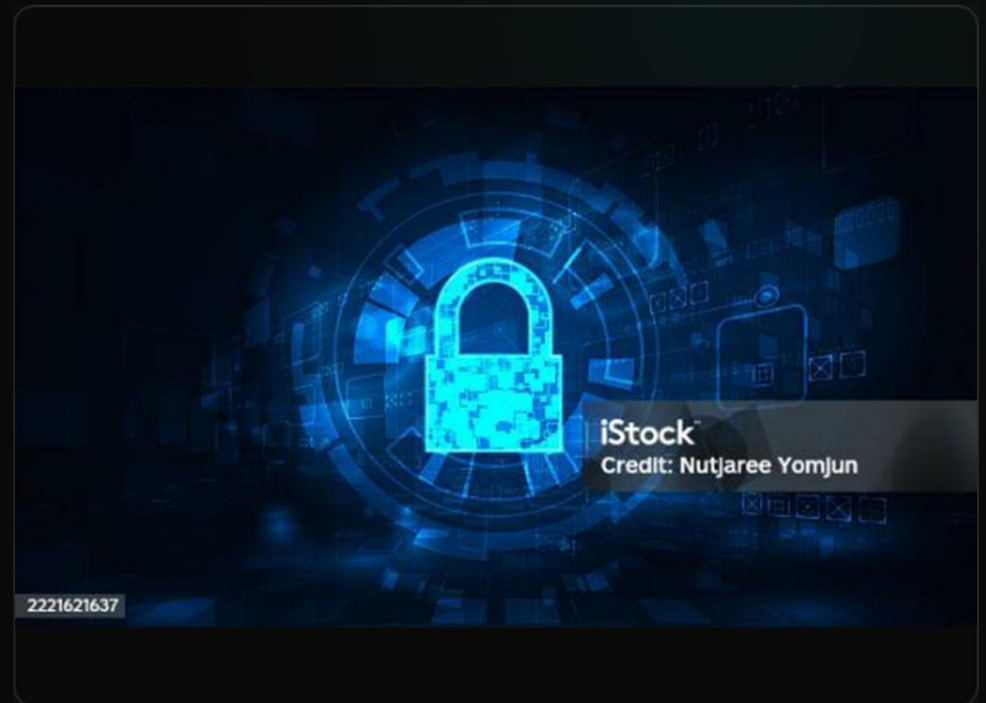
> Positions 설정

비밀번호 파라미터 값에만 페이로드 마커(\$)를 설정합니다.

password= \$ 1234 \$

> Payloads 로드

준비된 사전 파일(pass.txt)을 로드하거나 예상되는 비밀번호(123456, password 등)를 직접 입력하여 공격을 시작합니다.



Phase 2: 상태 코드 분석



기대 (Expectation)

일반적으로 로그인 성공하면 200 OK, 실패하면 401 Unauthorized나 403 Forbidden이 반환될 것으로 예상합니다.



현실 (Reality)

대부분의 웹 사이트는 실패 시에도 200 OK를 반환하며, 단순히 "Login failed"라는 텍스트만 출력합니다. 따라서 상태 코드만으로는 구분이 불가능합니다.

```
23735 2025-12-03 22:51:27.1314... 10.1.0.20 172.16.10.50 HTTP
23790 2025-12-03 22:51:29.9124... 172.16.10.50 10.1.0.20 HTTP
23793 2025-12-03 22:51:29.9155... 10.1.0.20 172.16.10.50 HTTP
23841 2025-12-03 22:51:32.7291... 172.16.10.50 10.1.0.20 HTTP
23844 2025-12-03 22:51:32.7321... 10.1.0.20 172.16.10.50 HTTP
23892 2025-12-03 22:51:35.5740... 172.16.10.50 10.1.0.20 HTTP
23895 2025-12-03 22:51:35.5774... 10.1.0.20 172.16.10.50 HTTP
23947 2025-12-03 22:51:39.4851... 172.16.10.50 10.1.0.20 HTTP
23950 2025-12-03 22:51:39.4899... 10.1.0.20 172.16.10.50 HTTP
23999 2025-12-03 22:51:42.3366... 172.16.10.50 10.1.0.20 HTTP
24002 2025-12-03 22:51:42.4690... 10.1.0.20 172.16.10.50 HTTP
24134 2025-12-03 22:51:45.3042... 172.16.10.50 10.1.0.20 HTTP
24137 2025-12-03 22:51:45.3077... 10.1.0.20 172.16.10.50 HTTP
24186 2025-12-03 22:51:48.3187... 172.16.10.50 10.1.0.20 HTTP
24189 2025-12-03 22:51:48.3223... 10.1.0.20 172.16.10.50 HTTP
24316 2025-12-03 22:51:51.3651... 172.16.10.50 10.1.0.20 HTTP
24319 2025-12-03 22:51:51.3677... 10.1.0.20 172.16.10.50 HTTP
24501 2025-12-03 22:51:54.4419... 172.16.10.50 10.1.0.20 HTTP
24504 2025-12-03 22:51:54.4504... 10.1.0.20 172.16.10.50 HTTP
24638 2025-12-03 22:51:57.0433... 172.16.10.50 10.1.0.20 HTTP
24641 2025-12-03 22:51:57.0464... 10.1.0.20 172.16.10.50 HTTP
24796 2025-12-03 22:52:00.1974... 172.16.10.50 10.1.0.20 HTTP

474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=654321&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
850 GET /DVWA/vulnerabilities/brute/?username=admin&password=michael&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=ashley&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=qwerty&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=111111&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=iloveu&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=000000&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
851 GET /DVWA/vulnerabilities/brute/?username=admin&password=michelle&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
849 GET /DVWA/vulnerabilities/brute/?username=admin&password=tigger&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
851 GET /DVWA/vulnerabilities/brute/?username=admin&password=sunshine&Login=Login HTTP/1.1
474 HTTP/1.1 200 OK (text/html)
852 GET /DVWA/vulnerabilities/brute/?username=admin&password=chocolate&Login=Login HTTP/1.1
```

| 핵심 분석: Content-Length (길이)

Request ^	Payload	Status code	Response rec...	Error	Timeout	Length	Comment
5	letmein	200	22			5062	
6	admin	200	30			5063	
7	root	200	30			5063	
8	password	200	34			5106	
9	qwerty	200	32			5063	
10	123456	200	31			5063	
11	test123	200	31			5063	
12	1q2w3e4r	200	31			5063	

수많은 응답 패킷 중 " 길이가 다른 하나" 가 바로 비밀번호를 맞춘 요청입니다.

Attempt 1

4635 Bytes (Fail)

Attempt 2

4635 Bytes (Fail)

Success

4680 Bytes (Login OK)

Attempt 4

4635 Bytes (Fail)

Attempt 5

4635 Bytes (Fail)

Phase 3: Deduction - 공격 판단 기준



요청 속도 (Speed)

http.log 또는 Access Log 분석 결과, 동일한 IP에서 1초에 수십 번씩 특정 URL에 POST 요청을 보냅니다.



User-Agent & Referer

Burp는 일반 브라우저 정보를 사용하므로 식별이 어렵지만, 로그인 시도가 반복되므로 Referer 헤더는 일정하게 유지됩니다.



최종 결론 (Criteria)

특정 로그인 URL(/vulnerabilities/brute/)에 대해 5초 이내에 10회 이상의 요청이 발생하면 공격으로 간주합니다.

심화 분석: Zeek Log & Length Filter

Snort 경보 발생 시, Zeek의 http.log를 분석하여 정확한 공격 성공 시점을 찾아낼 수 있습니다.

Time (ts)	Source IP	Method	URI	Response Length
14:00:01	192.168.1.50	POST	/vulnerabilities/brute/	4635
14:00:01	192.168.1.50	POST	/vulnerabilities/brute/	4635
14:00:02	192.168.1.50	POST	/vulnerabilities/brute/	4635
14:00:02	192.168.1.50	POST	/vulnerabilities/brute/	4635

| 요약 (Summary)

- > Response Length가 핵심 지표
웹 공격의 성공 여부는 Status Code(200 OK)가 아니라 응답 패킷 길이의 미세한 변화로 찾아내야 합니다.
- > Content & URI 분석
애플리케이션 계층(L7) 공격은 단순 포트 정보가 아닌, URL과 패킷 내용(Content)을 깊이 있게 분석해야 합니다.
- > 임계값 기반 탐지
Snort의 detection_filter 옵션을 사용하여 짧은 시간 내 발생하는 반복적인 요청을 효과적으로 차단할 수 있습니다.



Scenario 4 Analysis

시스템 명령어 주입

OS Command Injection

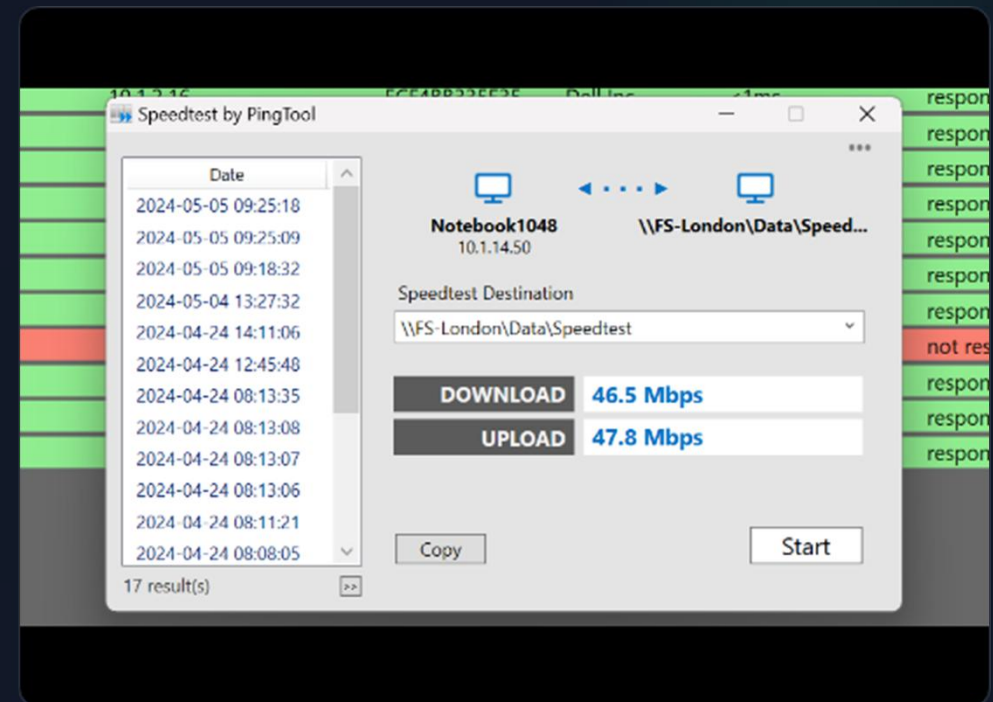
보안 위협 분석 및 Snort 대응 정책
수립 가이드

| Phase 1: 정상 기능 확인 (Baseline)

정상 Ping 테스트 절차

- > 접근: DVWA 로그인 > Command Injection 메뉴
- > 입력: IP Address란에 8.8.8.8 입력
- > 결과: 실제 Ping 명령어가 실행된 결과 출력

이는 서버 내부에서 ping 8.8.8.8과 같은 형태로 명령어가 실행되고 있음을 의미합니다.



| 공격 기법: 메타 문자(Meta Character) 활용

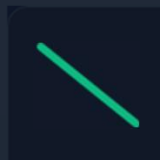
입력값 뒤에 특수 문자를 붙여 추가 명령어를 실행시키는 기법입니다.



Case A: 연결 연산자 (&&)

앞의 명령어가 성공하면 뒤의 명령어도 실행합니다.

```
8.8.8.8 && cat /etc/passwd
```



Case B: 파이프 (|)

앞 명령어의 결과를 뒤 명령어의 입력으로 넘깁니다.

```
| ls ; pwd
```



공격 목표

시스템 계정 정보(/etc/passwd) 확인 및 디렉터리 목록(ls) 조회 등을 통한 권한 탈취 시도.

Phase 2: 현상 관찰 및 패킷 분석

🔍 URL Encoding의 비밀

브라우저는 특수문자를 자동으로 인코딩하여 전송합니다.
Wireshark에서 Raw Packet을 확인하면 변환된 값을 볼 수 있습니다.

```
Raw: ip=8.8.8.8+%26%26+cat+%2Fetc...
```

```
Decoded: %26 → & (AND Operator)
```

```
Decoded: %2F → / (Directory Slash)
```

⚠️ Response Body (정보 유출)

서버의 응답 패킷(Response Body)을 살펴보면, 요청하지 않은 민감한 시스템 정보가 평문으로 노출된 것을 확인할 수 있습니다.

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin...
```

Phase 3: 추론 - 정상 vs 비정상 판별

"IP 주소를 입력하는 칸에 cat, ls, pwd 명령어나 &, | 같은 특수문자가 들어올 이유가 있을까요?"

✓ 정상 입력 (Normal)
숫자(0-9)와 점(.)으로만 구성됨.

(예: 192.168.1.1)

✗ 비정상 입력 (Abnormal)
알파벳, 공백, 특수문자가 섞여 있음.
(예: &&, cat, /etc/passwd)

ANALYZING NETWORK PERFORMANCE PARAMETERS USING WIRESHARK

Rachid Taki

Department of Computer & Information Technology, Jeddah Industrial College, Jeddah Industrial City, Kingdom of Saudi Arabia

ABSTRACT

Network performance can be a prime concern for network administrators. The performance of the network depends on many factors. Some of the issues found in the network performance are - slow Internet, bottlenecks, loss of packets and/or retransmissions, and excessive bandwidth consumption. For troubleshooting a network, an in-depth understanding of network protocols is required. The main objective of this research is to analyze the performance and various other parameters related to the capacity of a network in a home-based network environment using Wireshark. Network traffic is captured for different devices. The captured traffic is then analyzed using Wireshark's basic statistical tools and subsequent tools for various performance parameters.

KEYWORDS

Network traffic, sniffing, TCP, Wireshark, packet analyzer

1. INTRODUCTION

Now days the usage of the internet has increased everywhere. The use of the internet has become a necessity for everyone. While using the network there can be many network issues such as network slow, malwares infections, and slowing down of the internet. So network traffic monitoring becomes important to make sure that the network is running smoothly and efficiently. To monitor and analyze data traffic, a packet sniffer tool is used. Wireshark is a network packet analyzer. Packet sniffing is a technique for monitoring every packet that is sent over the network. A packet analyzer also known as a network analyzer, protocol analyzer, or packet sniffer is a computer program that can intercept and log traffic that passes over a digital network or part of a network and then presents captured packet data in as much detail as possible [1]. Packet sniffing is usually done by hackers or malicious insiders to carry out prohibited actions such as stealing passwords, and snooping other important data [2]. Packet sniffers are of two types: active sniffers and passive sniffers. Active sniffers can send data in the network and can be detected by other systems through different techniques whereas passive sniffers only collect data, but cannot be detected. Wireshark is a passive network sniffer [3]. Moreover, the structure of a packet sniffer consists of two parts: the packet analyzer which works on the application layer protocol, and the packet capture (pcap) which captures packets from all other layers [4]. The way packet sniffing works is divided into three processes: network collecting, conversion, analysis, and data theft [5].

Packet sniffing can be legitimately used by a network or system administrator to monitor and troubleshoot network traffic. Using the information captured by the packet sniffer an administrator can identify erroneous packets and use the data to pinpoint bottlenecks and help restore efficient network data transmission. The security threat presented by sniffers is their ability to capture all

Phase 4: Snort 탐지 정책 수립 전략

단순 빈도수(Threshold)가 아닌, 패킷의 내용(Content)을 정밀 검사하는 룰을 작성해야 합니다.

1. 정상 요청 허용 (Pass Rule)



숫자와 점으로만 구성된 패턴은 통과

2. Chaining Attack 탐지



명령어 연결 메타 문자(&, |, ;) 포함 시 경보

3. 민감한 파일 접근 탐지



/etc/passwd 등 시스템 파일 열람 시도 탐지



| Snort 탐지 룰 구현 (Code Implementation)

</>탐지 룰 예시 (Detection Rule)

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80 (  
  msg: "OS Command Injection Detected";  
  content: "|26 26|"; # && (Hex)  
  sid: 1000002;  
)
```

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80 (  
  msg: "Sensitive File Access";  
  content: "/etc/passwd";  
  sid: 1000003;  
)
```

검증 결과 (Verification)

- rule-update 명령어로 룰 적용
- 공격 수행 시 fast.log 확인

```
[Alert] OS Command Injection Detected  
[Alert] Sensitive File Access
```

두 가지 경보가 동시에 발생하여 공격의 의도를 명확히 파악할 수 있습니다.